Chapter 05

# 로지스틱 회귀 - 중급

Intermediate Logistic Regression

수리적 기초부터 논문 활용까지

대상: 박사과정 / 연구자 | 난이도: 중급 (기본 < 본 강의 < 확장)

기계학습 강의 시리즈

# 목차

1. [수리적 기초: 로지스틱 회귀의 유도](#1-수리적-기초-로지스틱-회귀의-유도)
- 1.1 일반화 선형 모형(GLM)에서의 위치
- 1.2 로짓 링크 함수와 시그모이드
- 1.3 최대우도추정(MLE) 유도
- 1.4 경사하강법과 뉴턴-랩슨법

2. [계수의 통계적 해석](#2-계수의-통계적-해석)
- 2.1 오즈비(Odds Ratio)의 심층 이해
- 2.2 Wald 검정과 신뢰구간
- 2.3 우도비 검정(Likelihood Ratio Test)
- 2.4 다중공선성과 VIF

3. [모델 평가의 심화](#3-모델-평가의-심화)
- 3.1 혼동 행렬과 파생 지표
- 3.2 ROC 곡선과 AUC
- 3.3 정밀도-재현율 곡선(PR Curve)
- 3.4 모델 보정(Calibration)
- 3.5 Hosmer-Lemeshow 적합도 검정

4. [정규화와 변수 선택](#4-정규화와-변수-선택)
- 4.1 L1 정규화 (Lasso)
- 4.2 L2 정규화 (Ridge)
- 4.3 Elastic Net
- 4.4 정규화 강도(C) 튜닝

5. [실전 실습: 타이타닉 데이터 심층 분석](#5-실전-실습-타이타닉-데이터-심층-분석)
- 5.1 고급 전처리와 피처 엔지니어링
- 5.2 통계적 모델 구축 (statsmodels)
- 5.3 sklearn 모델과 정규화 비교
- 5.4 ROC/AUC 분석
- 5.5 교차검증과 모델 안정성

6. [연구 관점: 논문에서의 로지스틱 회귀](#6-연구-관점-논문에서의-로지스틱-회귀)
- 6.1 로지스틱 회귀를 쓰는 연구 분야
- 6.2 논문에서 결과를 보고하는 방법
- 6.3 한계와 대안 모델

- 6.4 논문 주제 아이디어

---

# 1. 수리적 기초: 로지스틱 회귀의 유도

## 1.1 일반화 선형 모형(GLM)에서의 위치

로지스틱 회귀는 일반화 선형 모형(Generalized Linear Model, GLM) 의 한 특수 사례이다. GLM은 세 가지 요소로 구성된다:

| | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | - | - | - | - | \| | | | | | | | | | | | | | | | | | | | | | |
| n | t | ) | | \| | | 반 | 응 | 변 | 수 | | Y | 의 | | 분 | 포 | | \| | | 베 | 르 | 누 | 이 | | 분 | 포 | | ( |
| m | p | o | n | e | n | t | ) | | \| | | 선 | 형 | | 예 | 측 | 자 | | η | | = | | X | β | | \| | | η |
| | \| | | E | ( | Y | ) | 와 | | η | 를 | | 연 | 결 | | \| | | 로 | 짓 | | 함 | 수 | : | | l | o | g | i |

일반 선형 회귀(OLS)는 GLM에서 정규분포 + 항등 링크 함수를 사용하는 특수 사례이고, 로지스틱 회귀는 베르누이 분포 + 로짓 링크 함수를 사용하는 특수 사례이다.

GLM의 의의: 로지스틱 회귀를 GLM의 프레임워크 안에서 이해하면, 포아송 회귀(Poisson Regression), 프로빗 회귀(Probit Regression) 등 다른 분류/회귀 모형과의 관계를 체계적으로 파악할 수 있다. 논문에서 "왜 로지스틱 회귀를 선택했는가"를 정당화할 때 이 관점이 필요하다.

---

## 1.2 로짓 링크 함수와 시그모이드

### 로짓 변환의 수학적 구조

확률 p ∈ (0, 1)에 대해:

```
\text{logit}(p) = \ln\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 +
\cdots + \beta_p x_p
```

이 변환의 핵심은 확률 공간 (0,1)을 실수 전체 (-∞, +∞)로 매핑하는 것이다.

| \| | | 범 | 위 | | \| | | 변 | 환 | | \| | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | - | - | \| | - | - | - | - | - | - | \| | - | - | - | - | - | - | \| | | | | |
| | ( | 0 | , | | 1 | ) | | \| | | - | | \| | | | | | | | | | | |
| 1 | - | p | ) | | \| | | ( | 0 | , | | + | ∞ | ) | | \| | | 확 | 률 | | → | | 오 |
| p | / | ( | 1 | - | p | ) | ) | | \| | | ( | - | ∞ | , | | + | ∞ | ) | | \| | | 오 |

역변환(시그모이드)은:

$$p = \sigma(\eta) = \frac{1}{1 + e^{-\eta}} = \frac{e^{\eta}}{1 + e^{\eta}}$$

### 로짓 vs 프로빗 vs 보완 로그-로그

로지스틱 회귀 외에도 이진 반응변수를 다루는 링크 함수가 있다:

| | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | \| | - | - | - | - | - | - | \| | | | | | | | | | | | | | |
| \| | | l | n | ( | p | / | ( | 1 | - | p | ) | ) | | \| | | 대 | 칭 | , | | 오 | 즈 | 비 | |
| * | | \| | | Φ | | ¹ | ( | p | ) | | ( | 표 | 준 | 정 | 규 | | 역 | C | D | F | ) | | \| |
| g | l | o | g | ) | * | * | | \| | | l | n | ( | - | l | n | ( | 1 | - | p | ) | ) | | \| |

논문 Tip: 의학/역학 분야에서는 오즈비 해석이 관행이므로 로짓을 쓰고, 계량경제학에서는 프로빗도 자주 사용한다. 희귀 사건(rare event)에서는 보완 로그-로그를 고려할 수 있다.

---

## 1.3 최대우도추정 (Maximum Likelihood Estimation, MLE)

기본 강의에서는 "로그 손실을 최소화한다"고만 설명했다. 여기서는 왜 로그 손실이 나오는지를 MLE로부터 유도한다.

### 우도함수 (Likelihood Function)

n개의 독립 관측치 (x , y )에 대해, y ~ Bernoulli(p )이면:

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{1-y_i}$$

여기서 p = σ(x β)이다.

### 로그우도함수 (Log-Likelihood)

계산의 편의와 수치적 안정성을 위해 로그를 취한다:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left[ y_i \ln(p_i) + (1-y_i)\ln(1-p_i) \right]$$

로그 손실(Log Loss)은 음의 로그우도를 n으로 나눈 것이다:

$$\text{Log Loss} = -\frac{1}{n}\ell(\boldsymbol{\beta})$$

따라서 로그 손실을 최소화하는 것 = 로그우도를 최대화하는 것 = MLE이다.

### 그래디언트 (Gradient)

로그우도를 β에 대해 편미분하면:

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^{n} (y_i - p_i) x_{ij}$$

벡터 형태로:

$$\nabla_{\boldsymbol{\beta}} \ell = \mathbf{X}^T(\mathbf{y} - \mathbf{p})$$

이 그래디언트가 0이 되는 β를 찾으면 MLE를 얻는다. 하지만 닫힌 형태의 해(closed-form solution)가 존재하지 않으므로 반복적 수치 최적화가 필요하다.

### 헤시안 (Hessian)

2차 도함수(헤시안)는:

$$\mathbf{H} = \frac{\partial^2 \ell}{\partial \boldsymbol{\beta} \partial \boldsymbol{\beta}^T} = -\mathbf{X}^T \mathbf{W} \mathbf{X}$$

여기서 W = diag(p (1-p ))는 대각 가중치 행렬이다.

헤시안이 항상 음의 준정부호(negative semi-definite) 이므로, 로그우도는 오목(concave) 함수이다. 즉, 지역 최대값 = 전역 최대값이므로 경사상승법(또는 경사하강법으로 음의 로그우도 최소화)이 반드시 수렴한다.

논문 Tip: "로지스틱 회귀의 비용 함수는 볼록(convex)이므로 전역 최적해가 보장된다"는 점은 신경망 등 비볼록 모델과의 중요한 차별점이다.

---

## 1.4 최적화 알고리즘

### 경사하강법 (Gradient Descent)

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} + \alpha \cdot \mathbf{X}^T(\mathbf{y} - \mathbf{p}^{(t)})$$

- α: 학습률 (learning rate)
- 수렴 속도가 느릴 수 있음

### 뉴턴-랩슨법 (Newton-Raphson) / IRLS

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \mathbf{H}^{-1} \nabla \ell$$

이를 정리하면 반복 재가중 최소제곱법(IRLS: Iteratively Reweighted Least Squares) 이 된다:

$$\boldsymbol{\beta}^{(t+1)} = (\mathbf{X}^T \mathbf{W}^{(t)} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W}^{(t)} \mathbf{z}^{(t)}$$

여기서 z = Xβ + W ¹(y - p )는 조정된 반응변수(adjusted response)이다.

| r | n | | s | o | l | v | e | r | | \| | | 특 | 성 | | \| | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | - | - | - | - | - | - | - | - | - | - | \| | - | - | - | - | - | - | \| | | | | | | | | | | | | | | | | | | | |
| ' | s | g | d | ' | ` | | ( | S | G | D | C | l | a | s | s | i | f | i | e | r | ) | | \| | | 대 | 규 | 모 | | 데 | 이 | 터 | 에 | | 적 | 합 | | | | |
| b | f | g | s | ' | ` | | ( | 기 | 본 | 값 | ) | | \| | | 준 | 뉴 | 턴 | 법 | , | | 중 | 규 | 모 | | 데 | 이 | 터 | 에 | | 효 | 율 | 적 | | \| | | | | | |
| n | e | w | t | o | n | - | c | g | ' | ` | , | | ` | ' | n | e | w | t | o | n | - | c | h | o | l | e | s | k | y | ' | ` | | \| | | 수 | 렴 | | | |
| ' | s | a | g | ' | ` | , | | ` | ' | s | a | g | a | ' | ` | | \| | | 확 | 률 | 적 | | 평 | 균 | | 경 | 사 | , | | L | 1 | | 지 | 원 | ( | s | | | |

| | | |

```
# 1■■ ■■■
or_1unit = np.exp(coefs)

# Age■ 10■ ■■, ■■■■ 1■■
or_custom = or_1unit.copy()
or_custom['Age'] = np.exp(coefs['Age'] * 10)

print("=== ■■■ (1■■) ===")
print(or_1unit.round(4))
print("\n=== ■■■ (■■■ ■■) ===")
print(or_custom.round(4))
```

---

## 2.2 Wald 검정과 신뢰구간

MLE의 점근적 성질에 의해, β 은 대표본에서 근사적으로 정규분포를 따른다:

$$\hat{\beta}_j \sim N\left(\beta_j, \left[(\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1}\right]_{jj}\right)$$

### Wald 검정 (개별 계수의 유의성)

$$z_j = \frac{\hat{\beta}_j}{\text{SE}(\hat{\beta}_j)}, \quad z_j \sim N(0,1) \text{ under } H_0: \beta_j = 0$$

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | | | | |
| 5 | | \| | | 해 | 당 | | 피 | 처 | 가 | | 통 | 계 | |
| 0 | 5 | | \| | | 해 | 당 | | 피 | 처 | 의 | | 유 | |

### 오즈비의 95% 신뢰구간

$$\text{CI}_{95\%}(\text{OR}_j) = \exp\left(\hat{\beta}_j \pm 1.96 \times \text{SE}(\hat{\beta}_j)\right)$$

논문 Tip: 논문에서는 반드시 오즈비와 함께 95% 신뢰구간을 보고해야 한다. 신뢰구간이 1을 포함하면 해당 피처는 통계적으로 유의하지 않다.

### statsmodels를 활용한 상세 통계 결과

```
import statsmodels.api as sm

# statsmodels■ ■■■(intercept)■ ■■■■■ ■■■■ ■
X_with_const = sm.add_constant(X_train)

# ■■■■ ■■ (GLM with Binomial family)
logit_model = sm.Logit(y_train, X_with_const)
result = logit_model.fit()

# ■■ ■■ ■■
print(result.summary())

# ■■■■ 95% ■■■■
params = result.params
```

```
conf = result.conf_int()
conf.columns = ['2.5%', '97.5%']
conf['OR'] = np.exp(params)
conf['OR_2.5%'] = np.exp(conf['2.5%'])
conf['OR_97.5%'] = np.exp(conf['97.5%'])
conf['p-value'] = result.pvalues
print("\n=== ■■■ ■ 95% ■■■■ ===")
print(conf[['OR', 'OR_2.5%', 'OR_97.5%', 'p-value']].round(4))
```

statsmodels summary() 출력 해석:

| | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | \| | | | | | | | | | | | |
| | 회 | 귀 | | 계 | 수 | | ( | β | | ) | | \| | |
| * | | \| | | 표 | 준 | 오 | 차 | | S | E | ( | β | |
| l | d | | z | - | 통 | 계 | 량 | | \| | | | | |
| * | | \| | | p | - | v | a | l | u | e | | ( | |
| 0 | . | 9 | 7 | 5 | ] | * | * | | \| | | β | | |
| l | i | h | o | o | d | * | * | | \| | | 로 | 그 | |
| ² | * | * | | \| | | M | c | F | a | d | d | e | |

---

## 2.3 우도비 검정 (Likelihood Ratio Test, LRT)

Wald 검정은 개별 계수를 검정하지만, 모형 전체의 유의성이나 중첩 모형(nested models)의 비교에는 우도비 검정이 더 정확하다.

$$\Lambda = -2(\ell_{\text{reduced}} - \ell_{\text{full}}) \sim \chi^2_{df}$$

여기서 df = 풀 모형의 파라미터 수 - 축소 모형의 파라미터 수.

```
from scipy import stats

# ■ ■■ (■■ ■■)
full_model = sm.Logit(y_train, sm.add_constant(X_train)).fit(disp=0)

# ■■ ■■ (Age ■■)
X_reduced = X_train.drop('Age', axis=1)
reduced_model = sm.Logit(y_train, sm.add_constant(X_reduced)).fit(disp=0)

# ■■■ ■■
LR_stat = -2 * (reduced_model.llf - full_model.llf)
df = 1  # ■■■ ■■ ■
p_value = stats.chi2.sf(LR_stat, df)

print(f"LR ■■■: {LR_stat:.4f}")
print(f"■■■: {df}")
print(f"p-value: {p_value:.4f}")
```

논문 Tip: 변수 선택의 정당화에 우도비 검정을 사용하면 리뷰어를 설득하기 쉽다. "Age를 제거해도 모형의 적합도가 통계적으로 유의하게 나빠지지 않았다 (LRT, $\chi^2$ = 2.31, df = 1, p = 0.128)"와 같이 보고한다.

## 2.4 다중공선성과 VIF (Variance Inflation Factor)

다중공선성은 독립변수들 사이에 높은 상관관계가 존재할 때 발생하며, 계수 추정의 분산을 증가시킨다.

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

여기서 $R^2$ 는 x 를 나머지 독립변수로 회귀했을 때의 결정계수이다.

| | 값 | | | | 해 | 석 | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | - | - | \| | - | - | - | - | - | - | \| | | | | | | |
| | 다 | 중 | 공 | 선 | 성 | | 없 | 음 | | \| | | | | | | | |
| | \| | | 약 | 한 | | 다 | 중 | 공 | 선 | 성 | | ( | 보 | 통 | | 허 | 용 |
| 0 | | \| | | 중 | 간 | | 다 | 중 | 공 | 선 | 성 | | ( | 주 | 의 | | 필 |
| 0 | | \| | | 심 | 각 | 한 | | 다 | 중 | 공 | 선 | 성 | | ( | 조 | 치 | |

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

# VIF ■■
X_vif = sm.add_constant(X_train)
vif_data = pd.DataFrame()
vif_data['Feature'] = X_vif.columns
vif_data['VIF'] = [variance_inflation_factor(X_vif.values, i) for i in range(X_vif.shape[1])]
print(vif_data)
```

# 3. 모델 평가의 심화

## 3.1 혼동 행렬과 파생 지표

기본 강의에서는 정확도(Accuracy)만 다뤘다. 연구에서는 더 정교한 지표가 필요하다.

### 혼동 행렬 (Confusion Matrix)

| \| | 예 | 측 | : | 1 | ( | 양 | 성 | ) | \| | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| ( | T | r | u | e | N | e | g | a | t | i | v | e | ) | \| | F | P | ( | F | a | l | s | e | P | o | s | i | t | i | v | e | | |
| ( | F | a | l | s | e | N | e | g | a | t | i | v | e | ) | - | T | y | p | e | I | I | 오 | 류 | \| | T | P | ( | | | | | | |

### 주요 평가 지표

| \| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | - | - | - | - | - | - | - | - | \| | | | | | | | | | | | | | | | | | | | | | |
| P | + | T | N | ) | / | ( | T | P | + | T | N | + | F | P | + | F | N | ) | | \| | | 전 | 체 | | 정 | 답 | | 비 | 율 | | \| | |
| P | / | ( | T | P | + | F | P | ) | | \| | | 양 | 성 | | 예 | 측 | 의 | | 정 | 확 | 성 | | \| | | F | P | | 비 | 용 | 이 | | 클 |
| t | y | ) | * | * | | \| | | T | P | / | ( | T | P | + | F | N | ) | | \| | | 실 | 제 | | 양 | 성 | | 포 | 착 | 률 | | \| | |
| | T | N | / | ( | T | N | + | F | P | ) | | \| | | 실 | 제 | | 음 | 성 | | 포 | 착 | 률 | | \| | | T | N | 이 | | 중 | 요 | 할 |
| i | o | n | × | R | e | c | a | l | l | / | ( | P | r | e | c | i | s | i | o | n | + | R | e | c | a | l | l | ) | | \| | | 정 |

```
from sklearn.metrics import (
    confusion_matrix, classification_report,
    precision_score, recall_score, f1_score
)

# ■■ ■■
cm = confusion_matrix(y_test, pred)
print("=== ■■ ■■ ===")
print(cm)

# ■■ ■■■
print("\n=== Classification Report ===")
print(classification_report(y_test, pred, target_names=['■■(0)', '■■(1)']))
```

---

## 3.2 ROC 곡선과 AUC

ROC(Receiver Operating Characteristic) 곡선은 임계값을 0에서 1까지 변화시키면서 TPR(재현율) vs FPR(1-특이도) 를 그린 곡선이다. AUC(Area Under Curve) 는 이 곡선 아래의 면적으로, 모델의 판별력(discriminative ability)을 종합적으로 평가한다.

| | 값 | | \| | | 판 | 별 | 력 | | \| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | - | - | \| | - | - | - | - | - | - | - | - | \| | | | | |
| | \| | | 무 | 작 | 위 | | 추 | 측 | | 수 | 준 | | ( | 판 | 별 | 력 | | |
| - | 0 | . | 7 | | \| | | 약 | 한 | | 판 | 별 | 력 | | \| | | | |
| - | 0 | . | 8 | | \| | | 적 | 절 | 한 | | 판 | 별 | 력 | | \| | | |
| - | 0 | . | 9 | | \| | | 좋 | 은 | | 판 | 별 | 력 | | \| | | | |
| - | 1 | . | 0 | | \| | | 우 | 수 | 한 | | 판 | 별 | 력 | | \| | | |

### AUC의 확률적 해석

AUC는 "양성 샘플의 예측 확률이 음성 샘플의 예측 확률보다 높을 확률"과 같다:

$$\text{AUC} = P(\hat{p}_{\text{positive}} > \hat{p}_{\text{negative}})$$

```
from sklearn.metrics import roc_curve, roc_auc_score, auc
import matplotlib.pyplot as plt

# ROC ■■ ■■■
prob = model.predict_proba(X_test)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, prob)
roc_auc = auc(fpr, tpr)
```

```
# ■■■
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, 'b-', linewidth=2, label=f'ROC curve (AUC = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], 'r--', linewidth=1, label='Random classifier (AUC = 0.5)')
plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
plt.ylabel('True Positive Rate (Sensitivity)', fontsize=12)
plt.title('ROC Curve', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# ■■ ■■■ ■■ (Youden's J statistic)
J = tpr - fpr
optimal_idx = np.argmax(J)
optimal_threshold = thresholds[optimal_idx]
print(f"■■ ■■■ (Youden's J): {optimal_threshold:.4f}")
print(f"  TPR = {tpr[optimal_idx]:.4f}, FPR = {fpr[optimal_idx]:.4f}")
```

논문 Tip: 정확도 대신 AUC를 주요 성능 지표로 사용하는 것이 연구에서 표준이다. AUC는 임계값에 독립적이며, 불균형 데이터에서도 안정적이다.

---

## 3.3 정밀도-재현율 곡선 (PR Curve)

클래스 불균형이 심한 경우 (예: 양성 비율 < 10%), ROC-AUC는 지나치게 낙관적일 수 있다. 이때 PR 곡선이 더 정보를 잘 전달한다.

```
from sklearn.metrics import precision_recall_curve, average_precision_score

precision_arr, recall_arr, thresholds_pr = precision_recall_curve(y_test, prob)
ap = average_precision_score(y_test, prob)

plt.figure(figsize=(8, 6))
plt.plot(recall_arr, precision_arr, 'g-', linewidth=2, label=f'PR curve (AP = {ap:.3f})')
plt.xlabel('Recall', fontsize=12)
plt.ylabel('Precision', fontsize=12)
plt.title('Precision-Recall Curve', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

---

## 3.4 모델 보정 (Calibration)

모델이 "생존 확률 0.8"이라고 출력했을 때, 실제로 그 그룹에서 80%가 생존했는지를 확인하는 것이 보정(Calibration) 이다. 로지스틱 회귀는 다른 모델(Random Forest, SVM 등)에 비해 보정이 잘 되는 편이다.

```
from sklearn.calibration import calibration_curve

# ■■ ■■
fraction_of_positives, mean_predicted_value = calibration_curve(
    y_test, prob, n_bins=10
)
```

```
plt.figure(figsize=(8, 6))
plt.plot(mean_predicted_value, fraction_of_positives, 's-', label='Logistic Regression')
plt.plot([0, 1], [0, 1], 'k--', label='Perfectly calibrated')
plt.xlabel('Mean Predicted Probability', fontsize=12)
plt.ylabel('Fraction of Positives', fontsize=12)
plt.title('Calibration Curve (Reliability Diagram)', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

논문 Tip: 의료/금융처럼 확률 자체가 의사결정에 사용되는 도메인에서는 보정 곡선을 반드시 보고해야 한다.

---

## 3.5 Hosmer-Lemeshow 적합도 검정

모델이 데이터를 잘 설명하는지를 통계적으로 검정한다. 예측 확률을 10개 그룹(decile)으로 나누고, 각 그룹의 관찰 빈도와 기대 빈도를 비교한다.

$$\hat{C} = \sum_{g=1}^{G} \frac{(O_g - E_g)^2}{E_g(1 - E_g/n_g)} \sim \chi^2_{G-2}$$

- H : 모형이 데이터에 적합하다
- p-value > 0.05이면 적합도가 양호

```
from scipy import stats

def hosmer_lemeshow_test(y_true, y_prob, n_groups=10):
    """Hosmer-Lemeshow ■■■ ■■"""
    data = pd.DataFrame({'y': y_true, 'prob': y_prob})
    data['group'] = pd.qcut(data['prob'], n_groups, duplicates='drop')

    observed = data.groupby('group')['y'].sum()
    expected = data.groupby('group')['prob'].sum()
    n_per_group = data.groupby('group')['y'].count()

    hl_stat = ((observed - expected)**2 / (expected * (1 - expected / n_per_group))).sum()
    df = len(observed) - 2
    p_value = stats.chi2.sf(hl_stat, df)

    return hl_stat, df, p_value

hl_stat, df, p_val = hosmer_lemeshow_test(y_test.values, prob)
print(f"Hosmer-Lemeshow ■■: χ² = {hl_stat:.4f}, df = {df}, p = {p_val:.4f}")
if p_val &gt; 0.05:
    print("→ ■■■ ■■■■ ■■■■ (■■■■ ■■ ■■)")
else:
    print("→ ■■■ ■■■■ ■■■■ (■■■■ ■■)")
```

---

# 4. 정규화와 변수 선택

## 4.1 L2 정규화 (Ridge)

기본 로지스틱 회귀의 비용 함수에 L2 페널티를 추가한다:

```
J(\boldsymbol{\beta}) = -\ell(\boldsymbol{\beta}) +
\frac{\lambda}{2}\|\boldsymbol{\beta}\|_2^2 = -\ell(\boldsymbol{\beta}) +
\frac{\lambda}{2}\sum_{j=1}^{p}\beta_j^2
```

- 계수를 0에 가깝게 축소(shrink) 하지만, 정확히 0으로 만들지는 않음
- 다중공선성이 있을 때 계수 추정을 안정화
- sklearn에서 `penalty='l2'` (기본값)

## 4.2 L1 정규화 (Lasso)

```
J(\boldsymbol{\beta}) = -\ell(\boldsymbol{\beta}) +
\lambda\|\boldsymbol{\beta}\|_1 = -\ell(\boldsymbol{\beta}) +
\lambda\sum_{j=1}^{p}|\beta_j|
```

- 일부 계수를 정확히 0으로 만들어 자동 변수 선택 효과
- 고차원 데이터(p >> n)에서 유용
- sklearn에서 `penalty='l1'`, `solver='saga'` 또는 `'liblinear'`

## 4.3 Elastic Net

L1과 L2의 혼합:

```
J(\boldsymbol{\beta}) = -\ell(\boldsymbol{\beta}) +
\lambda\left[\rho\|\boldsymbol{\beta}\|_1 +
\frac{1-\rho}{2}\|\boldsymbol{\beta}\|_2^2\right]
```

- ρ = l1_ratio: L1과 L2의 비율 (0 = 순수 L2, 1 = 순수 L1)
- 상관된 피처 그룹에서 L1보다 안정적
- sklearn에서 `penalty='elasticnet'`, `solver='saga'`

## 4.4 정규화 강도(C) 튜닝

sklearn의 `LogisticRegression`에서 `C = 1/λ`이다. C가 클수록 정규화가 약하고, 작을수록 강하다.

```
from sklearn.model_selection import cross_val_score
import numpy as np

# C ■■ ■■ ■■■■ ■■ ■■
C_values = [0.001, 0.01, 0.1, 1, 10, 100]
results = []

for C in C_values:
    model_reg = LogisticRegression(C=C, penalty='l2', solver='lbfgs', max_iter=1000)
    scores = cross_val_score(model_reg, X_train, y_train, cv=5, scoring='roc_auc')
    results.append({
```

```
        'C': C, 'mean_auc': scores.mean(), 'std_auc': scores.std()
    })
    print(f"C={C:7.3f}: AUC = {scores.mean():.4f} (+/- {scores.std():.4f})")

# ■■■
results_df = pd.DataFrame(results)
plt.figure(figsize=(8, 5))
plt.semilogx(results_df['C'], results_df['mean_auc'], 'bo-', linewidth=2)
plt.fill_between(
    results_df['C'],
    results_df['mean_auc'] - results_df['std_auc'],
    results_df['mean_auc'] + results_df['std_auc'],
    alpha=0.2
)
plt.xlabel('C (Inverse of regularization strength)', fontsize=12)
plt.ylabel('Mean AUC (5-fold CV)', fontsize=12)
plt.title('Regularization Strength vs. Model Performance', fontsize=14)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

### L1 정규화를 통한 변수 선택

```
# L1 ■■■■ ■■ ■■
model_l1 = LogisticRegression(penalty='l1', solver='saga', C=0.1, max_iter=5000)
model_l1.fit(X_train, y_train)

coef_l1 = pd.Series(model_l1.coef_[0], index=X.columns)
print("=== L1 ■■■ ■■ (C=0.1) ===")
print(coef_l1.round(4))
print(f"\n0■ ■■ ■■■ ■: {(coef_l1 != 0).sum()} / {len(coef_l1)}")
print(f"■■■ ■■: {coef_l1[coef_l1 != 0].index.tolist()}")
```

---

# 5. 실전 실습: 타이타닉 데이터 심층 분석

## 5.1 고급 전처리와 피처 엔지니어링

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, roc_auc_score, roc_curve,
    classification_report, confusion_matrix
)
from sklearn.preprocessing import StandardScaler
import statsmodels.api as sm
import warnings
warnings.filterwarnings('ignore')

# ========================================
# ■■■ ■■
# ========================================
file_url = 'https://media.githubusercontent.com/media/musthave-ML10/data_source/main/titanic.csv'
```

```
data = pd.read_csv(file_url)

print("=== ■■ ■■■ ===")
print(data.head())
print(f"\n■■■ ■■: {data.shape}")

# ========================================
# ■■ ■■ ■■■■■
# ========================================

# 1) Name■■ ■■(Title) ■■
data['Title'] = data['Name'].str.extract(r' ([A-Za-z]+)\.')
title_mapping = {
    'Mr': 'Mr', 'Miss': 'Miss', 'Mrs': 'Mrs', 'Master': 'Master',
    'Dr': 'Rare', 'Rev': 'Rare', 'Col': 'Rare', 'Major': 'Rare',
    'Mlle': 'Miss', 'Ms': 'Miss', 'Mme': 'Mrs', 'Don': 'Rare',
    'Dona': 'Rare', 'Lady': 'Rare', 'Countess': 'Rare',
    'Jonkheer': 'Rare', 'Sir': 'Rare', 'Capt': 'Rare'
}
data['Title'] = data['Title'].map(title_mapping).fillna('Rare')
print("\n=== ■■ ■■ ===")
print(data['Title'].value_counts())

# 2) ■■ ■■ ■■ ■■
data['FamilySize'] = data['SibSp'] + data['Parch'] + 1  # ■■ ■■
data['IsAlone'] = (data['FamilySize'] == 1).astype(int)

# 3) ■■ ■■■ (Binning)
data['AgeBin'] = pd.cut(data['Age'], bins=[0, 12, 18, 35, 60, 80],
                        labels=['Child', 'Teen', 'Young', 'Middle', 'Senior'])

# 4) ■■■■ ■■ ■■
data_clean = data.drop(['Name', 'Ticket'], axis=1)

# 5) ■■ ■■ ■■
data_model = pd.get_dummies(data_clean,
                            columns=['Sex', 'Embarked', 'Title', 'AgeBin'],
                            drop_first=True)

# ■■ Age■ ■■, AgeBin■ ■■ ■■■ ■■
print(f"\n=== ■■ ■■ ■: {data_model.shape[1] - 1}■ ===")
print(data_model.columns.tolist())
```

## 5.2 통계적 모델 구축 (statsmodels)

```
# ========================================
# statsmodels■ ■■■ ■■
# ========================================

# ■■/■■ ■■
feature_cols = [c for c in data_model.columns if c != 'Survived']
X = data_model[feature_cols]
y = data_model['Survived']

# ■■/■■■ ■■
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# statsmodels Logit
X_train_sm = sm.add_constant(X_train)
X_test_sm = sm.add_constant(X_test)
```

```
logit_result = sm.Logit(y_train, X_train_sm).fit()
print(logit_result.summary())

# ■■■ + 95% CI ■■■
print("\n" + "="*60)
print("■■■ ■ 95% ■■■■")
print("="*60)
params = logit_result.params
conf = logit_result.conf_int()
or_table = pd.DataFrame({
    'Coef': params,
    'OR': np.exp(params),
    'OR_CI_Lower': np.exp(conf[0]),
    'OR_CI_Upper': np.exp(conf[1]),
    'p-value': logit_result.pvalues
})
print(or_table.round(4))

# ■■■ ■■■ ■■■
sig_vars = or_table[or_table['p-value'] &lt; 0.05]
print(f"\n■■■ ■■ (p &lt; 0.05): {len(sig_vars)}■")
print(sig_vars.round(4))
```

## 5.3 sklearn 모델과 정규화 비교

```
# ========================================
# sklearn: ■■■ ■■
# ========================================

# ■■■ (■■■ ■ ■■■■ ■■)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ■■ ■■
models = {
    'No Regularization': LogisticRegression(penalty=None, solver='lbfgs', max_iter=5000),
    'L2 (C=1.0)': LogisticRegression(penalty='l2', C=1.0, solver='lbfgs', max_iter=5000),
    'L2 (C=0.1)': LogisticRegression(penalty='l2', C=0.1, solver='lbfgs', max_iter=5000),
    'L1 (C=1.0)': LogisticRegression(penalty='l1', C=1.0, solver='saga', max_iter=5000),
    'L1 (C=0.1)': LogisticRegression(penalty='l1', C=0.1, solver='saga', max_iter=5000),
}

print("=== ■■■ ■■■ ■■ ■■ ===")
print(f"{'Model':25s} {'Accuracy':&gt;10s} {'AUC':&gt;10s} {'Non-zero':&gt;10s}")
print("-" * 57)

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    pred = model.predict(X_test_scaled)
    prob = model.predict_proba(X_test_scaled)[:, 1]
    acc = accuracy_score(y_test, pred)
    auc_val = roc_auc_score(y_test, prob)
    n_nonzero = (model.coef_[0] != 0).sum()
    print(f"{name:25s} {acc:10.4f} {auc_val:10.4f} {n_nonzero:10d}")
```

## 5.4 ROC/AUC 분석

```
# ========================================
# ROC ■■ ■■
# ========================================
```

```
plt.figure(figsize=(10, 7))

for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    prob = model.predict_proba(X_test_scaled)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, prob)
    auc_val = roc_auc_score(y_test, prob)
    plt.plot(fpr, tpr, linewidth=2, label=f'{name} (AUC={auc_val:.3f})')

plt.plot([0, 1], [0, 1], 'k--', linewidth=1, label='Random')
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curves: Regularization Comparison', fontsize=14)
plt.legend(fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

## 5.5 교차검증과 모델 안정성

```
# ========================================
# Stratified K-Fold ■■■■
# ========================================
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)

model_cv = LogisticRegression(penalty='l2', C=1.0, solver='lbfgs', max_iter=5000)
cv_auc = cross_val_score(model_cv, X_train_scaled, y_train, cv=skf, scoring='roc_auc')
cv_acc = cross_val_score(model_cv, X_train_scaled, y_train, cv=skf, scoring='accuracy')

print("=== 10-Fold Stratified Cross-Validation ===")
print(f"AUC: {cv_auc.mean():.4f} (+/- {cv_auc.std():.4f})")
print(f"Accuracy: {cv_acc.mean():.4f} (+/- {cv_acc.std():.4f})")
print(f"\n■ Fold■ AUC: {cv_auc.round(4)}")

# ■■■
fig, ax = plt.subplots(figsize=(8, 5))
ax.boxplot([cv_acc, cv_auc], labels=['Accuracy', 'AUC'])
ax.set_title('10-Fold CV Performance Distribution', fontsize=14)
ax.set_ylabel('Score', fontsize=12)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

---

# 6. 연구 관점: 논문에서의 로지스틱 회귀

## 6.1 로지스틱 회귀를 쓰는 연구 분야

| | 활 | 용 | | \| | | 논 | 문 | | 예 | 시 | | 주 | 제 | | \| | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| - | - | - | - | - | - | - | - | - | \| | - | - | - | - | - | - | - | - | - | - | - | - | - | - | \| | | | | | | | |
| * | | \| | | 질 | 병 | | 위 | 험 | | 요 | 인 | | 분 | 석 | | \| | | " | B | M | I | 와 | | 2 | 형 | | 당 | 뇨 | | 발 | 병 |

| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | \| | | 이 | 항 | | 결 | 과 | 의 | | 요 | 인 | | 분 | 석 | | \| | | " | 교 | 육 | | 수 | 준 | 이 | | 투 | 표 | | 참 | 여 | 에 |
| \| | | 고 | 객 | | 행 | 동 | | 예 | 측 | | \| | | " | 고 | 객 | | 이 | 탈 | | 예 | 측 | | 모 | 델 | 링 | | 및 | | 주 | 요 | |
| | 신 | 용 | | 평 | 가 | | \| | | " | 대 | 출 | | 부 | 도 | | 예 | 측 | 과 | | 차 | 별 | | 변 | 수 | | 영 | 향 | | 평 | 가 | " |
| | 학 | 업 | | 성 | 취 | | 예 | 측 | | \| | | " | 중 | 도 | | 탈 | 락 | | 위 | 험 | | 요 | 인 | | 식 | 별 | " | | \| | | |
| | 사 | 건 | | 발 | 생 | | 예 | 측 | | \| | | " | 산 | 불 | | 발 | 생 | | 확 | 률 | 에 | | 대 | 한 | | 기 | 상 | | 요 | 인 | |

## 6.2 논문에서 결과를 보고하는 방법

### 필수 보고 항목

1. 모델 설명: 사용한 모델, 정규화 여부, solver, 하이퍼파라미터
2. 변수 기술 통계: 각 독립변수의 평균, 표준편차, 범위
3. 다중공선성 진단: VIF 테이블
4. 주요 결과 테이블: 계수, 오즈비, 95% CI, p-value
5. 모델 적합도: Pseudo $R^2$, Log-Likelihood, AIC/BIC
6. 판별력: AUC with 95% CI, 교차검증 결과
7. 보정: Hosmer-Lemeshow 검정 또는 보정 곡선

### 논문 결과 테이블 형식 예시

```
Table 2. Logistic Regression Results for Titanic Survival Prediction

Variable          Coef (β)    OR      95% CI          p-value
═══════════════════════════════════════════════════════════════
Pclass            -1.182      0.307   [0.219, 0.430]  &lt;0.001 ***
Age (per 10y)     -0.399      0.671   [0.523, 0.861]   0.002 **
Sex_male          -2.569      0.077   [0.045, 0.131]  &lt;0.001 ***
FamilySize        -0.156      0.856   [0.741, 0.988]   0.034 *
Embarked_Q        -0.079      0.924   [0.512, 1.668]   0.793
Embarked_S        -0.236      0.790   [0.521, 1.198]   0.267
═══════════════════════════════════════════════════════════════
Model fit: Pseudo R² = 0.342, AUC = 0.862 [0.831, 0.893]
Hosmer-Lemeshow: χ²(8) = 7.23, p = 0.512

* p &lt; 0.05, ** p &lt; 0.01, *** p &lt; 0.001
```

### 결과 서술 예시

"로지스틱 회귀 분석 결과, 성별(Sex_male: OR = 0.077, 95% CI [0.045, 0.131], $p < 0.001$)이 생존 여부에 가장 강한 영향을 미치는 것으로 나타났다. 남성의 생존 오즈는 여성에 비해 약 92.3% 낮았다. 객실 등급(Pclass: OR = 0.307, 95% CI [0.219, 0.430], $p < 0.001$)도 유의한 예측 변수로, 등급이 한 단계 낮아질 때마다 생존 오즈가 69.3% 감소하였다. 모델의 판별력은 AUC = 0.862로 양호하였으며, Hosmer-Lemeshow 검정에서 적합도가 충분한 것으로 확인되었다 ($\chi^2(8) = 7.23$, $p = 0.512$)."

## 6.3 한계와 대안 모델

| | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| \| | | | | | | | | | | | | | | | | | | | |
| 에 | 서 | | 선 | 형 | 으 | 로 | 만 | | 분 | 류 | | 가 | 능 | | \| | | S | V | M |
| 로 | | 상 | 호 | 작 | 용 | 을 | | 학 | 습 | 하 | 지 | | 않 | 음 | | \| | | 상 | 호 |
| L | 1 | | 없 | 이 | 는 | | 성 | 능 | | 저 | 하 | | \| | | E | l | a | s | t |
| 가 | 정 | | 위 | 반 | | 시 | | 성 | 능 | | 저 | 하 | | \| | | G | A | M | , |
| 치 | 에 | | 영 | 향 | | 받 | 음 | | \| | | 로 | 버 | 스 | 트 | | 회 | 귀 | , | |

### 로지스틱 회귀의 장점 (대안 대비)

1. 해석 가능성(Interpretability): 오즈비로 직접 해석 가능 (XAI 불필요)
2. 통계적 추론: p-value, 신뢰구간, 우도비 검정 등 풍부한 추론 도구
3. 확률 보정: 자연스럽게 잘 보정된 확률 출력
4. 소규모 데이터: 적은 데이터에서도 안정적 (딥러닝보다 유리)
5. 규제 요건: 의료, 금융 등에서 설명 가능 모델 요구 시 적합

## 6.4 논문 주제 아이디어

로지스틱 회귀를 중심으로 쓸 수 있는 연구 방향:

### A. 방법론 연구

1. "불균형 데이터에서 로지스틱 회귀의 정규화 전략 비교"
- SMOTE + L1/L2/Elastic Net 조합의 성능을 체계적으로 비교
- 의료 데이터(희귀 질환)에 적용
2. "고차원 로지스틱 회귀에서의 변수 선택: Lasso vs Stability Selection"
- 유전체(genomics) 데이터에서 유의한 SNP를 찾는 문제
- 변수 선택의 안정성을 Stability Selection으로 개선
3. "Penalized Logistic Regression과 XGBoost의 해석 가능성-성능 트레이드오프"
- 다양한 도메인 데이터셋에서 두 모델의 성능과 해석력 비교
- SHAP을 활용한 XGBoost의 사후 해석과 로지스틱 회귀의 내재적 해석 비교

### B. 응용 연구

4. "COVID-19 중증화 예측을 위한 로지스틱 회귀 기반 위험도 스코어링"
- 임상 변수로 중증 전이 확률을 예측하는 스코어카드 개발
- Nomogram으로 시각화하여 임상 현장 활용
5. "온라인 학습 환경에서 학생 이탈 예측: 로지스틱 회귀와 학습 분석"

- LMS 로그 데이터를 피처로 활용
- 조기 개입(early intervention)을 위한 임계값 최적화

6. "소셜 미디어 텍스트의 감성 분류: TF-IDF + 로지스틱 회귀 파이프라인"

- 대규모 텍스트 데이터에서 L1 정규화의 피처 선택 효과 분석
- BERT 등 대형 모델과의 성능-비용 비교

---

# 핵심 요약 테이블

| | | |